UNIVERSITY OF RWANDA

COLLEGE OF SCIENCE AND TECHNOLOGY

NYARUGENGE CAMPUS

COMPUTER AND SOFTWARE ENGINEERING

YEAR 3

REG NO: 223021200

ON 24th MARCH, 2026

MOBILE APPLICATION SYSTEMS AND DESIGN

FLUTTER LAB5 REPORT

Offline Posts Manager

Local Database Integration, CRUD Operations & Stateful UI

Project	Offline Posts Manager
Framework	Flutter (Dart)
Storage	SharedPreferences (JSON)
Platform	Flutter Web (Chrome)
Architecture	Feature-First MVC
Operations	Create, Read, Update, Delete

1. Project Overview

The Offline Posts Manager is a Flutter application developed for a small media company whose staff need to manage posts without requiring an internet connection. The application provides a complete offline-first experience, allowing staff to create, view, edit, and delete posts entirely on-device with no network dependency.

1.1 Application Objectives

- Allow staff to view all posts stored locally on the device
- Enable reading the full details of any individual post
- Provide a form to add new posts with title, author, category, and content
- Allow editing of any existing post with change-tracking
- Enable permanent deletion of posts with confirmation dialogs

1.2 Project Architecture

The project follows a clean feature-first architecture separating concerns across four layers:

```
lib/
  main.dart → App entry point and theme configuration
  database/
    database_helper.dart→  Storage singleton with all CRUD operations
  models/
    post.dart → Post data model with toMap / fromMap
  screens/
    home_screen.dart → View all posts with search
    post_detail_screen.dart → Read full post details
    add_post_screen.dart →  Create new post
    edit_post_screen.dart  → Update existing post
```

2. Application Screenshots

The following screenshots were captured during development and testing of the Offline Posts Manager application running in Flutter Web (Chrome browser).

Screen 1 · No Posts Yet — Empty State

When the application is launched for the very first time, or after all posts have been deleted, the Home Screen displays an empty state. This is the default view before any content has been created. The empty state screen communicates clearly to the user that no posts are currently stored locally. Rather than showing a blank screen.

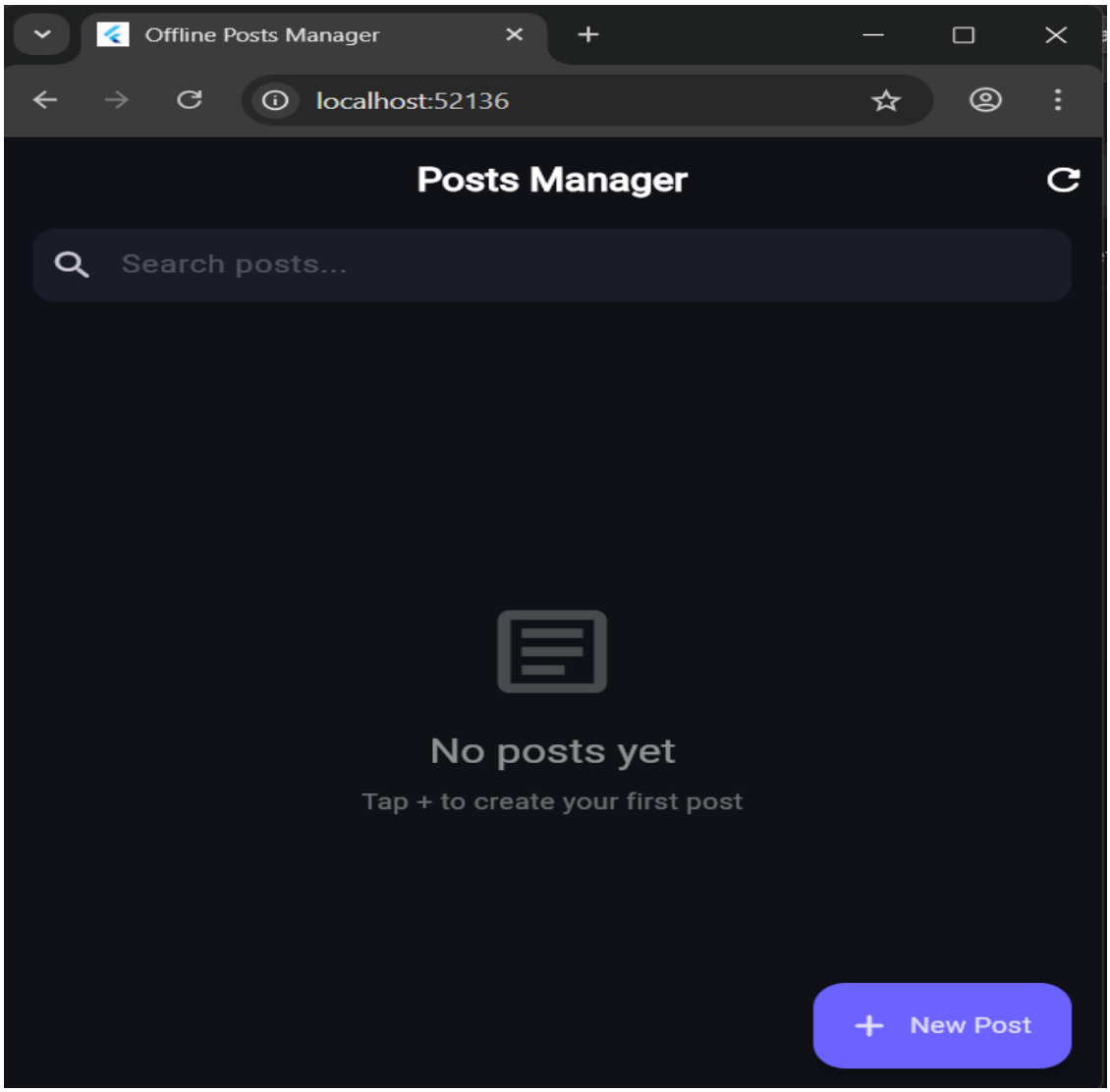
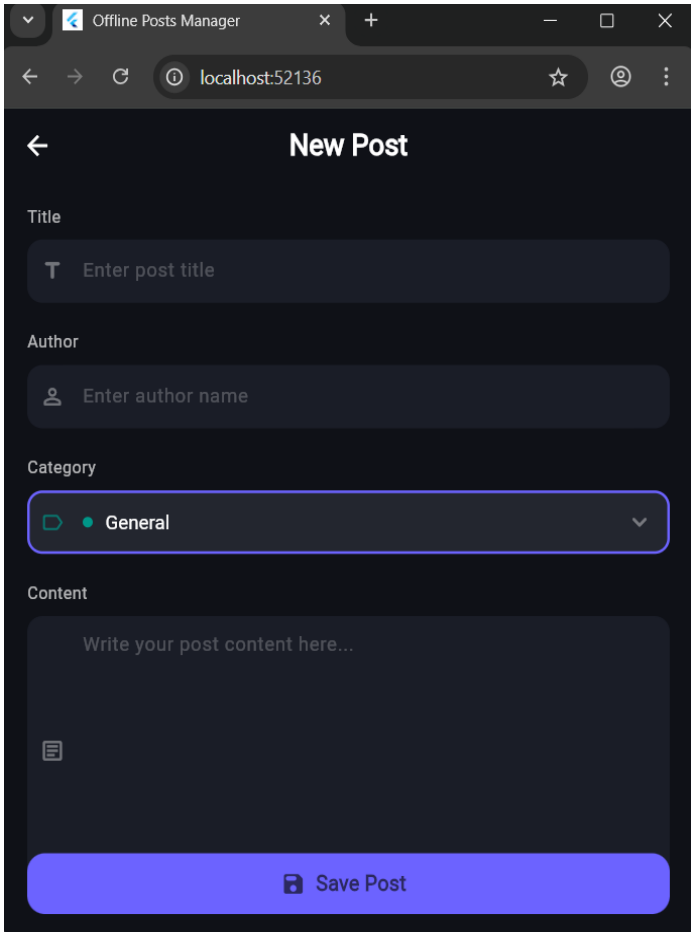


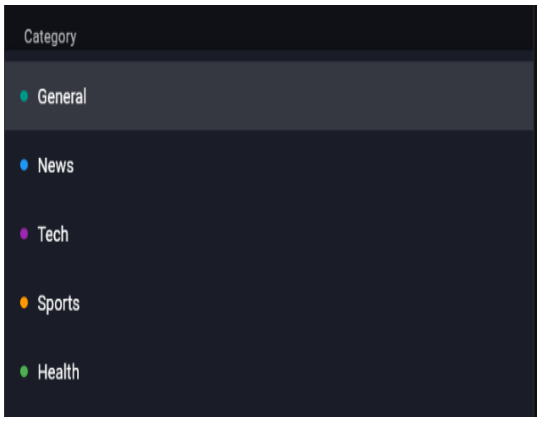
Figure 1: *Home Screen displaying the empty state when no posts are stored locally.*

Screen 2 · New Post — Create Post Screen

The New Post screen allows staff members to compose and save a brand-new post to local storage. It is reached by tapping the 'New Post' floating action button on the Home Screen.

All four fields (Title, Author, Category, Content) must pass validation before saving proceeds.





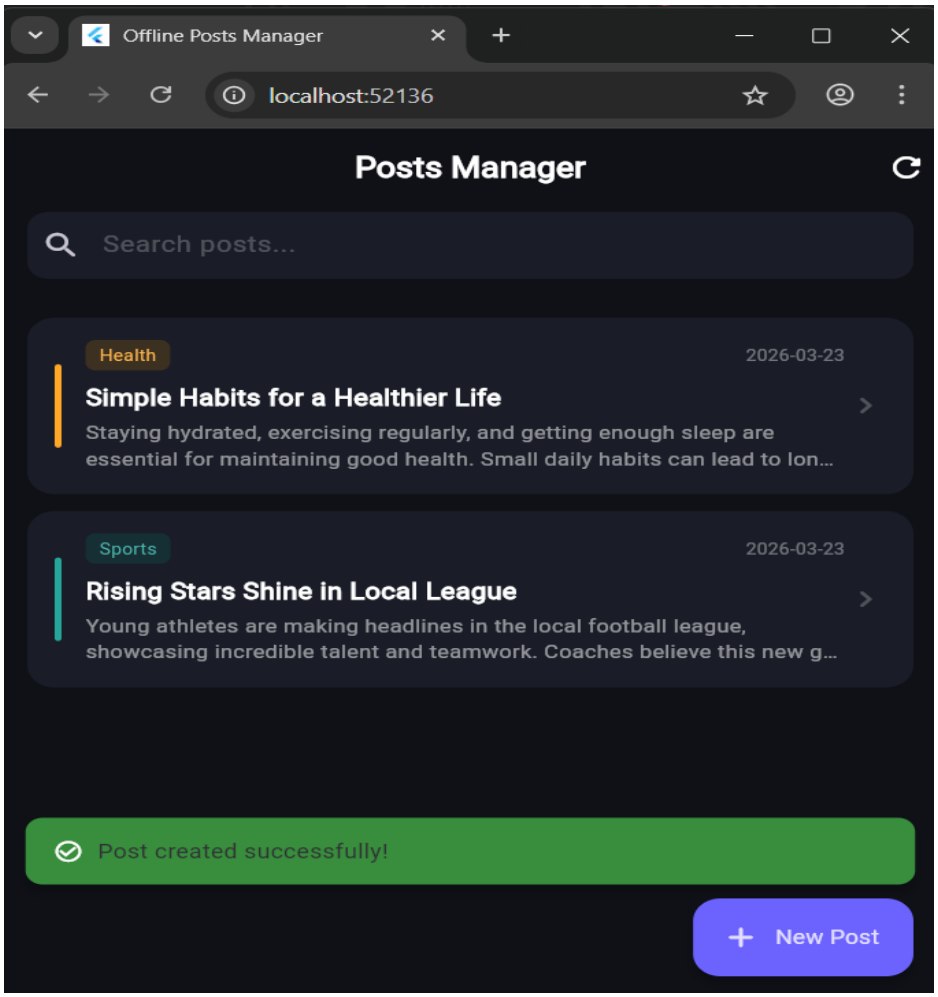


Figure 2: *New Post screen showing the create form with all fields and the Save Post button.*

Screen 3 · Edit Post — Update Post Screen

The Edit Post screen allows staff members to modify the content of an existing post. It is reached by tapping the edit icon (pencil) in the AppBar of the Post Detail screen.

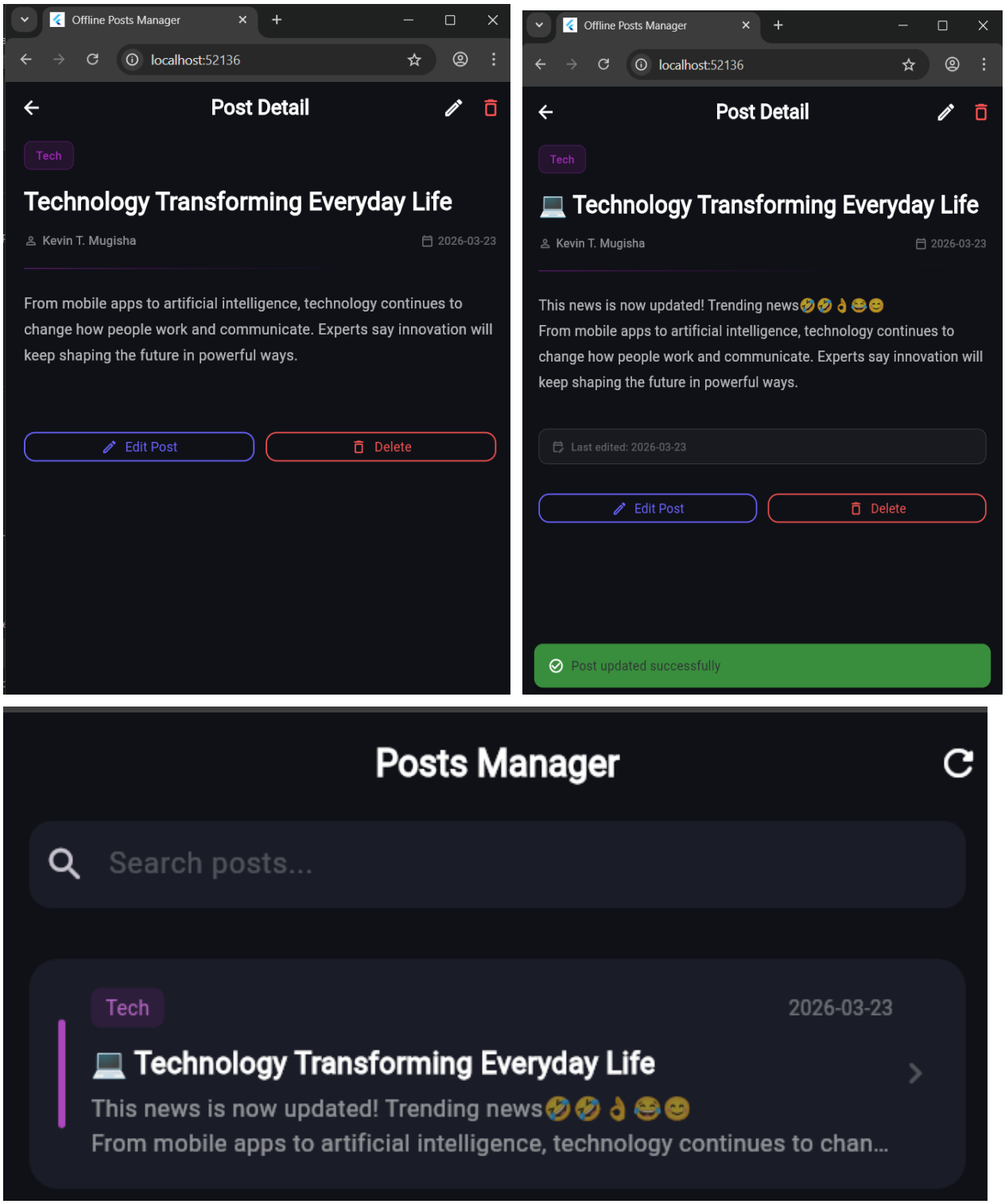


Figure 3: *Edit Post screen with all fields pre-populated from the existing post data.*

Screen 4 · Delete Post — Confirmation Dialog

The Delete Post confirmation is a modal dialog that appears when the staff member taps the delete icon (red bin) in the AppBar or the 'Delete' button at the bottom of the Post Detail screen.

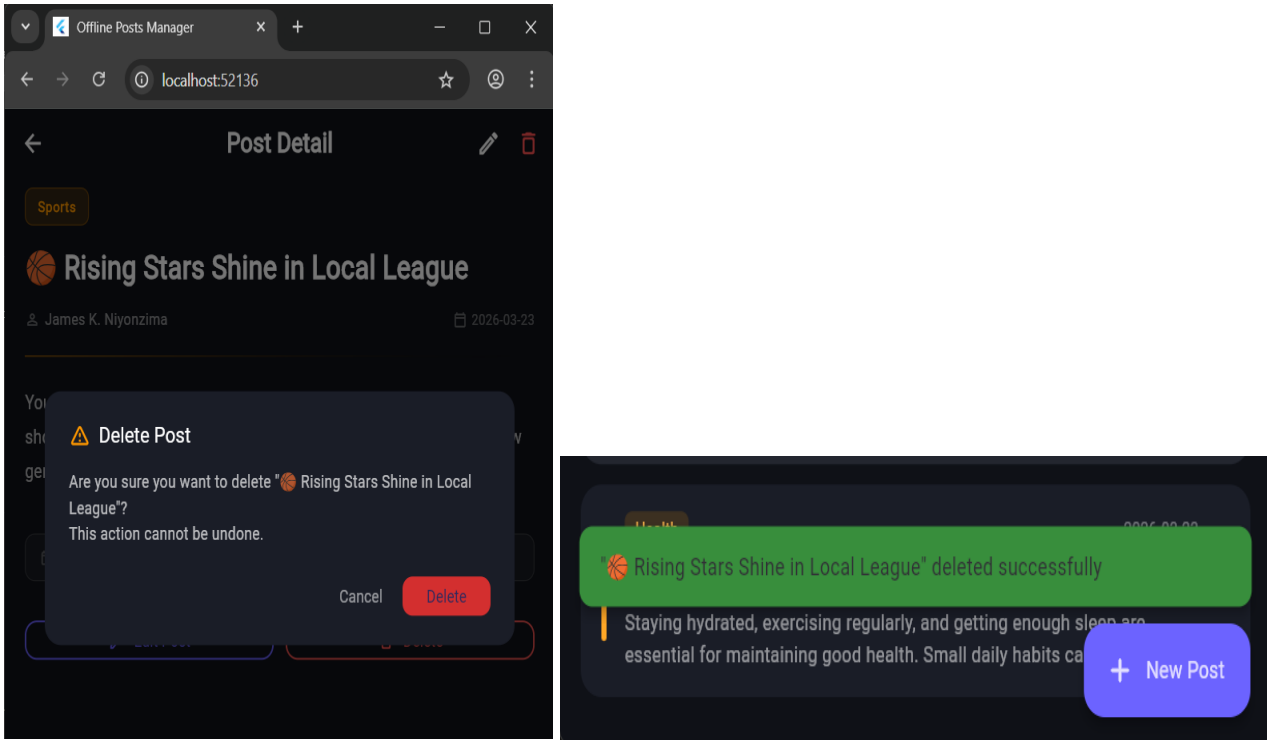


Figure 4: *Delete confirmation dialog with Cancel and Delete options preventing accidental data loss.*

Screen 5 · Search — Find Posts

The Search feature is embedded directly into the Home Screen's AppBar as a persistent text field. Staff can filter posts in real time without navigating to a separate screen.

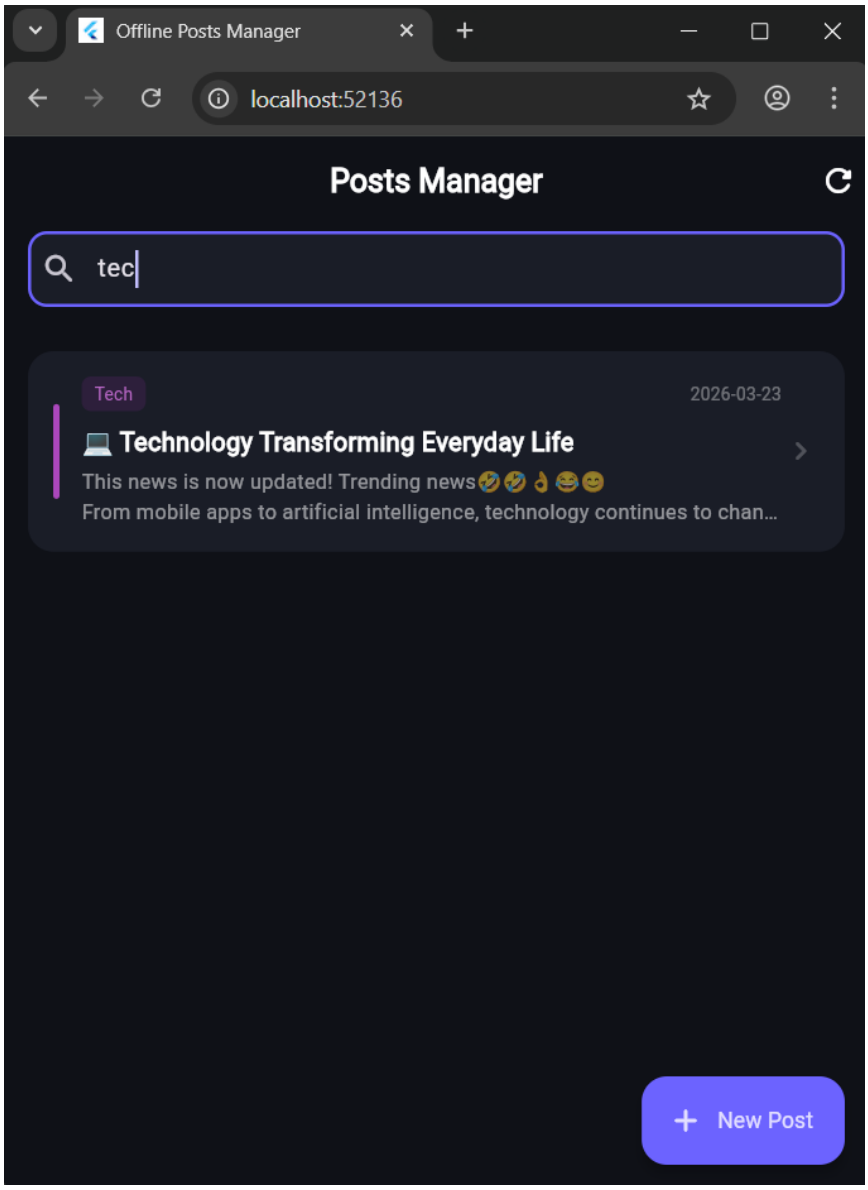


Figure 5: *Search field filtering posts in real time, showing only matching results.*

Screen 6 · Posts List — Home Screen with All Posts

The Posts List is the main Home Screen of the application, displayed whenever one or more posts are stored locally. It is the first screen the user sees after posts have been created and the central hub from which all other screens are accessed. This shows the edited posts and excludes the deleted posts.

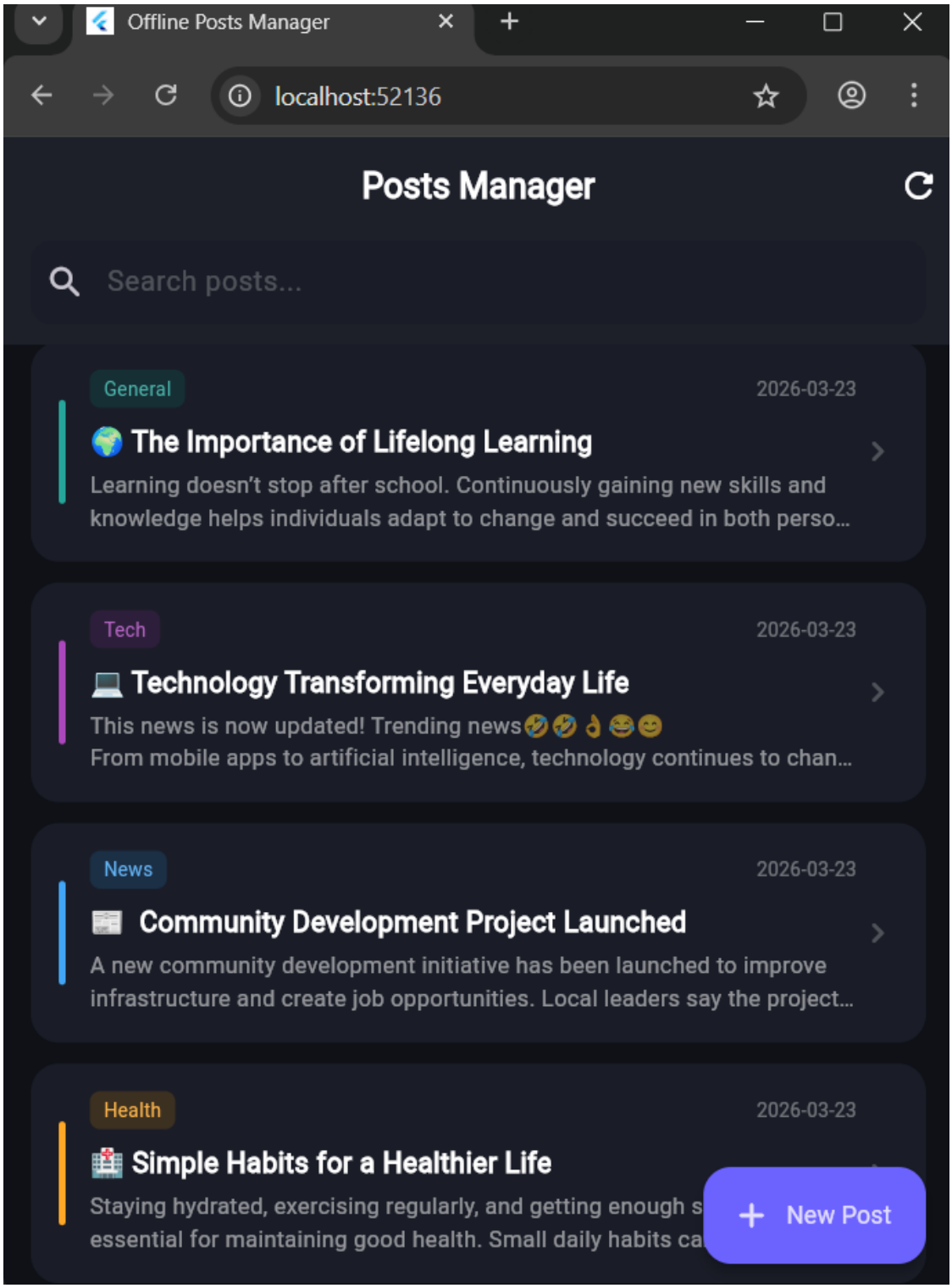


Figure 6: *Home Screen displaying all locally stored posts as interactive cards ordered newest-first.*

4. Conclusion

The Offline Posts Manager demonstrates effective local data storage, CRUD operations, and responsive UI management in Flutter. It meets all required features, including viewing, adding, editing, and deleting posts. The project highlights the importance of selecting the right storage solution—using SQLite for mobile platforms and SharedPreferences for web. By applying async/await and a two-layer error handling approach, the app remains smooth and stable. Overall, the project provided practical experience in handling real-world challenges and managing asynchronous data efficiently.

5. References

1. Flutter Documentation. (2024). SharedPreferences Plugin. https://pub.dev/packages/shared_preferences
2. Flutter Documentation. (2024). sqflite Plugin for Flutter. https://pub.dev/packages/sqflite
3. Google. (2024). Flutter — Build apps for any screen. https://flutter.dev
4. SQLite Consortium. (2024). SQLite Documentation. https://www.sqlite.org/docs.html
5. Dart Team. (2024). Dart Language — Asynchronous programming. https://dart.dev/codelabs/async-await

Question 1

* Dependencies used :

> Shared_preferences (Version ^2.2.2) : This is used to store data locally in Key-Value format, stores posts as JSON strings and works across web, Android and iOS with no complex setup.

> uuid (Version ^4.3.3) : It generates unique IDs for each post. It also replaces auto-increment IDs.

> flutter (SDK) : It provides UI components, navigations and state management.

> cupertino_icons : Provides iOS-style icons for better UI design.

* Why SQLite is necessary for local storage? SQLite is important because : > It works offline.
> It stores structured data in tables.
> It supports SQL queries.
> It ensures data integrity.

Question 2.

How to handle database exceptions?

a) Handling database not initialized.

```
final db= await database;
```

Here, database is initialized only when it is needed. If it fails, exception is thrown and caught in UI.

- Throw errors : adding meaning to errors and passe them to UI
- Catch errors : Preventing app crash and showing user-friendly message.

b) Handling insert / update / delete errors.

```
try {
  await db.insert(...);
} catch (e) {
  throw Exception("Insert failed: $e");
}
```

→ This prevents crashes, and logs and reports errors.

c) Handling invalid or corrupted data.

```
if (title.isEmpty || body.isEmpty) {
  throw Exception("Invalid data!");
}
```

→ Errors are caught and handled gracefully. And, App shows empty state instead of crashing.

Question 3.

SQLite in Flutter is a lightweight local database used to store structured data on device without internet.

* Database vs table.

A database is a file that stores all data. (eg: posts.db) while a table is a structure inside a database.

A database is identified by its file path on disk whereas a table is identified by its name (eg: posts). It has rows and columns.

* CRUD operations.

Create → db.insert() → Add post.
Read → db.query() → View posts.
Update → db.update() → Edit posts
Delete → db.delete() → Remove posts

* How Flutter interacts with the database asynchronously?

Flutter uses async / await to run database operations in the background, keeping the UI responsive. The app continues working smoothly while data is processed, and updates once the result is returned.